BUILDING AI INFRASTRUCTURE · PART 1 OF 7

Part 1: The Building Blocks

Chapter summary for beginners

In one sentence: every AI platform, at any company and on any cloud, is made of the same 8 building blocks. Learn them once and you can read any AI platform you'll ever see.

Video length: about 14 minutes · **Code:** none, this part is all ideas · **Cost:** $0

What AI infrastructure is

Every AI product does five things: it takes in **data**, **trains** a model, saves the **model**, **serves** it (answers requests), and **people** or other systems use those answers.

AI infrastructure is everything underneath those five steps: the machines, storage, network, tools and rules that make them run reliably, safely and at a fair cost.

> **The key idea:** the model is about 10% of the work. The other 90% is infrastructure. That's where the jobs are.

The 8 blocks, as a restaurant kitchen

#	Block	In the kitchen	What it does
1	**Compute**	the stove	CPUs run everyday work. GPUs do the heavy math for training and large models.
2	**Data and storage**	the pantry	Object storage holds raw data, training sets and versioned model files.
3	**Networking**	hallways and doors	A private network, one public front door (a load balancer), and everything else kept private.
4	**Orchestration**	the head chef	Kubernetes decides which container runs where, restarts crashes and adds copies when busy.
5	**MLOps**	the recipe book	Pipelines train the same way every time, a registry remembers every version, and gates block worse models.
6	**Serving**	the serving window	The model becomes an API that scales with demand. New versions go out slowly (canary).
7	**Observability**	the inspector	Metrics, logs and traces; SLOs define "good enough"; alerts fire only when users are affected.
8	**Guardrails**	locks, keys, budget	Least-privilege identity, secrets kept in a vault, and cost watched every day.

Each block, in plain words

1 · Compute. GPUs are the most expensive part of the platform. Good AI infrastructure shares them and turns them off when nobody is using them. We do that with a **GPU node pool that scales from zero**: it has no machines until GPU work arrives.

2 · Data and storage. A trained model is just a file with a version number. Keep fast storage close to the GPUs, because a GPU waiting for data is money burning.

3 · Networking. Only the front door is public. Your data, databases and GPUs should never be reachable from the internet.

4 · Orchestration. A **container** packs an app with everything it needs, so it runs the same everywhere. **Kubernetes** runs containers for you. This block is the heart of the series.

5 · MLOps. Training once by hand is a science project. Training the same way every time, automatically, is MLOps. **A worse model should never replace a better one by accident.**

6 · Serving. A **canary release** sends 10% of traffic to the new version first. If it's healthy, the new version gets the rest. If not, you roll back and almost nobody notices.

7 · Observability. An **SLO** (service level objective) is a reliability promise, such as "99.9% of requests succeed". Alert on what users feel, not on every small blip.

8 · Guardrails. Guardrails aren't the last step. They go around everything from day one.

Where each block lives

Block	AWS	Azure	Google Cloud	On-prem
Compute	EC2 GPU (G5, P5)	NC / ND VMs	A2 / G2 VMs	NVIDIA GPU servers
Storage	S3	Blob Storage	Cloud Storage	MinIO or Ceph
Network	VPC + LB	VNet + LB	VPC + LB	VLANs, firewall, MetalLB
Orchestration	EKS	AKS	GKE	OpenShift / Kubernetes
MLOps	SageMaker or MLflow + Kubeflow	Azure ML or MLflow + Kubeflow	Vertex AI or MLflow + Kubeflow	MLflow + Kubeflow
Serving	KServe / vLLM	KServe / vLLM	KServe / vLLM	KServe / vLLM
Observability	CloudWatch + Prometheus	Azure Monitor + Prometheus	Cloud Monitoring + Prometheus	Prometheus, Grafana, OTel
Guardrails	IAM, Secrets Manager, Budgets	Entra ID, Key Vault, Cost Mgmt	IAM, Secret Manager, Budgets	RBAC, Vault, OpenCost

The picture to remember

Compute, data and network sit at the bottom. Kubernetes sits on top of them. MLOps and serving run on Kubernetes. Observability watches everything from above, and guardrails wrap around all of it.

> **Only the bottom layer changes from cloud to cloud.** That's the design idea behind the whole series.

Try it (optional, free, about 5 minutes)

This repo has a tiny "ask the platform" demo that runs on your laptop with no cloud:

```
make doctor                                  # checks your tools
make start                                   # terminal 1: starts the demo API
make ask Q="what does the head chef do?"     # terminal 2: ask a question
```

Before Part 2

1. Install Docker, kubectl, Terraform and Git (`make doctor` tells you what's missing).
2. Create free accounts on AWS, Azure and Google Cloud.
3. **Set a budget alert on each one (for example $10)** before you create anything.
4. **Request GPU quota early.** New accounts start at zero GPUs, and approval can take days.

Words to know

- **GPU:** a processor that does thousands of small calculations at once, which is ideal for AI.
- **Container:** an app packed with everything it needs to run.
- **Kubernetes:** software that runs and manages containers across many machines.
- **Registry (model):** a catalog of every model version with its data, settings and score.
- **SLO:** a measurable reliability target, such as "99.9% of requests succeed".
- **Least privilege:** give each person or app only the access it needs.

Check yourself

1. Which block is "the head chef", and what does it decide?
2. Why should GPUs live in a pool that can shrink to zero?
3. What is a canary release?
4. Which layer changes when you move from AWS to Azure?

Answers: 1) Orchestration (Kubernetes): which container runs where, on which machine. 2) GPUs are the most expensive part, and scaling to zero means you pay only while they work. 3) Sending a small share of traffic (for example 10%) to a new version before all of it. 4) Only the bottom layer: compute, data and network.

Next: Part 2, where you build all eight blocks on your laptop for free.